KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

PROJECT TITLE

Submitted by

SUMISHA S

25BIT057

Under the Guidance of

DHILEEBAN A

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled **“ The Freelancer Project Management and Payment Tracking System”** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a **record of my original work** carried out under the guidance of **DHILEEBAN A**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

SUMISHA S

CERTIFICATE

This is to certify that the project entitled **" The Freelancer Project Management and Payment Tracking System "** is a **bona fide work** carried out by **SUMISHA S , B.Sc.I.T I-'A', Information Technology**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date: 09.04.2026

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The Freelancer Project Management and Payment Tracking System is a Java-based application designed to help freelancers manage their clients, projects, and payments in a structured and efficient manner. Freelancers often face challenges in keeping track of multiple clients, project statuses, and pending payments. Manual tracking using spreadsheets or notes can lead to errors, missed deadlines, and financial mismanagement.

This system provides a centralized platform where freelancers can add clients, assign projects, update project statuses, record payments, and track pending payments. The application is developed using Core Java concepts such as classes, objects, methods, loops, conditional statements, and exception handling.

The main objective is to simplify freelance work management, reduce manual errors, and ensure timely tracking of payments. This project also demonstrates the practical application of object-oriented programming principles in solving real-world business problems.

INTRODUCTION

Freelancing has become a popular career choice, offering flexibility and independence. However, managing multiple clients and projects simultaneously can be challenging. Freelancers often struggle with organizing client details, monitoring project progress, and ensuring timely payments. Without a proper system, this can lead to confusion, missed deadlines, and financial instability.

The Freelancer Project Management and Payment Tracking System addresses these challenges by providing a digital solution. Built using Core Java, the system allows freelancers to:

- **Add and manage client information.**
- **Create and monitor projects with status updates.**
- **Record payments and track pending amounts.**
- **Generate clear records of financial transactions.**

Objectives

- To design and develop a Java-based freelancer management system.
- To automate the process of client, project, and payment tracking.
- To reduce manual errors and improve efficiency.
- To provide a user-friendly interface for freelancers.
- To demonstrate the practical application of Java programming concepts.

Features

- Client Management (Add/View Clients)
- Project Management (Add/View Projects, Update Status)
- Payment Tracking (Add/View Payments, Pending Payments)
- Simple menu-driven interface
- Fast and efficient record management

PROGRAM ANALYSIS

Problem Definition

Freelancers often rely on manual methods such as spreadsheets or notebooks to manage clients, projects, and payments. These methods are prone to errors, lack organization, and make it difficult to track pending payments.

Project Overview

The system provides a menu-driven Java application that allows freelancers to manage clients, projects, and payments in one place. It ensures that all records are stored digitally, reducing the risk of data loss and improving accessibility.

Scope

- Applicable for freelancers across industries (IT, design, writing, consulting).
- Can be extended to include invoice generation and reporting.

- **Provides a foundation for integrating advanced features like database connectivity and online payment integration.**

Limitations

- **Currently console-based; GUI or web-based versions can be developed later.**
- **No database connectivity in the basic version; records are stored in memory only.**
- **Limited to basic project and payment tracking without advanced analytics.**

SYSTEM DESIGN

System design defines the structure and workflow of the application. The system is modular, with each module responsible for a specific function.

Modules

1. Client Module – Add and view client details.

2. Project Module – Add and view projects, update project status.

3. Payment Module – Add and view payments, track pending payments.

4. Exit Module – Allows safe termination of the application.

METHODOLOGY

The methodology outlines the step-by-step process followed during development.

1. Problem Analysis – Identified inefficiencies in manual freelance management.

2. System Design – Divided the system into modules for clarity and maintainability.

3. Program Development – Implemented using Core Java with OOP principles.

4. Testing and Validation – Tested with multiple clients, projects, and payments to ensure accuracy.

5. Result and Documentation – System successfully demonstrated automation of freelance management tasks.

Algorithm

1. Start program.

2. Display menu options.

3. Add/View Clients.

4. Add/View Projects.

5. Add/View Payments.

6. Track Pending Payments.

7. Exit program.

Flowchart

Start → Display Menu → Add Client → Add Project → Add Payment → View Pending Payments → End

SOURCE CODE

```
import java.util.*;


class Client {
  int clientId;
  String name;
  String email;


Client(int clientId, String name, String email) {
this.clientId = clientId;
this.name = name;
this.email = email;
}
}
```

```
class Project {
    int projectId;
    String title;
    String status;
    int clientId;

    Project(int projectId, String title, String status, int clientId) {
        this.projectId = projectId;
        this.title = title;
        this.status = status;
        this.clientId = clientId;
    }
}

class Payment {
    int paymentId;
    int projectId;
    double amount;
    String status;

    Payment(int paymentId, int projectId, double amount, String status) {
        this.paymentId = paymentId;
        this.projectId = projectId;
        this.amount = amount;
        this.status = status;
```

```
}
}

public class FreelancerSystem {

static Scanner sc = new Scanner(System.in);

static ArrayList<Client> clients = new ArrayList<>();
static ArrayList<Project> projects = new ArrayList<>();
static ArrayList<Payment> payments = new ArrayList<>();


public static void addClient() {
System.out.print("Enter Client ID: ");
int id = sc.nextInt();
sc.nextLine();

System.out.print("Enter Name: ");
String name = sc.nextLine();

System.out.print("Enter Email: ");
String email = sc.nextLine();

clients.add(new Client(id, name, email));
System.out.println("□ Client Added Successfully!\n");
}
```

```
public static void viewClients() {
System.out.println("\n--- Client List ---");
for (Client c : clients) {
System.out.println("ID: " + c.clientId +
" Name: " + c.name +
" Email: " + c.email);
}
}

public static void addProject() {
System.out.print("Enter Project ID: ");
int id = sc.nextInt();
sc.nextLine();

System.out.print("Enter Project Title: ");
String title = sc.nextLine();

System.out.print("Enter Status (Ongoing/Completed): ");
String status = sc.nextLine();

System.out.print("Enter Client ID: ");
int clientId = sc.nextInt();
```

```
projects.add(new Project(id, title, status, clientId));
System.out.println("□ Project Added Successfully!\n");
}

public static void viewProjects() {
System.out.println("\n--- Project List ---");
for (Project p : projects) {
System.out.println("ID: " + p.projectId +
" Title: " + p.title +
" Status: " + p.status +
" ClientID: " + p.clientId);
}
}

public static void addPayment() {
System.out.print("Enter Payment ID: ");
int id = sc.nextInt();

System.out.print("Enter Project ID: ");
int projectId = sc.nextInt();

System.out.print("Enter Amount: ");
double amount = sc.nextDouble();
sc.nextLine();
```

```
System.out.print("Enter Status (Paid/Pending): ");
String status = sc.nextLine();

payments.add(new Payment(id, projectId, amount, status));
System.out.println("□ Payment Recorded!\n");
}

public static void viewPayments() {
System.out.println("\n--- Payment List ---");
for (Payment p : payments) {
System.out.println("Payment ID: " + p.paymentId +
" Project ID: " + p.projectId +
" Amount: " + p.amount +
" Status: " + p.status);
}
}

public static void viewPendingPayments() {
System.out.println("\n--- Pending Payments ---");
for (Payment p : payments) {
if (p.status.equalsIgnoreCase("Pending")) {
System.out.println("Payment ID: " + p.paymentId +
" Project ID: " + p.projectId +
```

```
                " Amount: " + p.amount);
            }
        }
    }

    public static void main(String[] args) {

        int choice;

        do {
            System.out.println("\n===== Freelancer Management System =====");
            System.out.println("1. Add Client");
            System.out.println("2. View Clients");
            System.out.println("3. Add Project");
            System.out.println("4. View Projects");
            System.out.println("5. Add Payment");
            System.out.println("6. View Payments");
            System.out.println("7. View Pending Payments");
            System.out.println("8. Exit");

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {
                case 1:
```

```
addClient();
break;

case 2:
viewClients();
break;

case 3:
addProject();
break;

case 4:
viewProjects();
break;

case 5:
addPayment();
break;

case 6:
viewPayments();
break;

case 7:
viewPendingPayments();
break;
```

```
case 8:
System.out.println("Exiting... Thank You!");
break;

default:
System.out.println("Invalid choice!");
}

} while (choice != 8);
}
}
```

Discussion

The Freelancer Project Management and Payment Tracking System was developed to address the challenges faced by freelancers in managing clients, projects, and payments. The system provides a structured way to record client details, assign projects, monitor project statuses, and track payments. During development, Core Java concepts such as classes, objects, methods, loops, and conditional statements were applied effectively.

The system was tested with multiple inputs to ensure accuracy. For example, adding multiple clients and projects demonstrated that the system can handle multiple records simultaneously. Payment tracking was validated by recording both paid and pending transactions. The results confirmed that the system reduces manual effort, improves organization, and enhances financial transparency for freelancers.

Conclusion

The project successfully demonstrates how Java programming can be applied to solve real-world problems in freelance management. By automating client, project, and payment tracking, the system reduces errors, saves time, and ensures better financial management.

The project can be further enhanced by:

- **Integrating a database for permanent storage of records.**
- **Developing a graphical user interface (GUI) using Java Swing or JavaFX.**
- **Adding invoice generation and reporting features.**
- **Incorporating online payment gateways for real-time transactions.**

This project not only benefits freelancers but also serves as a learning tool for students to understand how programming concepts can be applied to business applications.

REFERENCES

- **Herbert Schildt –** ***Java: The Complete Reference***
- **Oracle Java Documentation – https://www.oracle.com/java**

- **GeeksforGeeks – https://www.geeksforgeeks.org**
    - **W3Schools – https://www.w3schools.com**
    - **Javatpoint – https://www.javatpoint.com**